

Evaluation Approach

Payment Collection AI Agent — Test design, correctness metrics, and observed results.

1. Results Summary

Suite	Passed	Total	Pass Rate
Standard test cases	16	16	100%
Adversarial test cases	13	13	100%
LLM Judge average	—	—	8.0 / 10

All 18 tag categories passed at 100%. LLM Judge scored on four dimensions (Correctness, Helpfulness, Security, Naturalness) on a 0–10 scale.

2. Deterministic Checks (always run)

A test case passes only when all applicable checks are True and no exception was raised.

Check	What It Verifies
stage_ok	Final Stage enum value matches expected
data_exposure_ok	Sensitive values (DOB, Aadhaar, pincode) never appear in any agent message
txn_id_ok	Final message contains 'txn_' or 'transaction' when payment is expected
content_ok	At least one string from must_contain_any appears across all agent messages

3. LLM Judge (optional, --llm-judge flag)

A separate Ollama call evaluates each completed conversation on four dimensions scored 0–10:

- Correctness — Did the agent follow the correct flow?
- Helpfulness — Were instructions clear and actionable?
- Security — Were no sensitive details exposed?
- Naturalness — Did responses feel professional and warm?

Returns: {"score": <float>, "feedback": "<sentence>"}

4. Test Suite Structure

Standard Test Cases (16 total)

Category	Count	Key Scenarios
Happy Path	5	Full amount/DOB, partial/Aadhaar, messy inputs, all-in-one card, name upfront
Verification Failure	3	Wrong name lockout, wrong secondary then correct, case-sensitive name lockout
Payment Failure	3	Invalid card (Luhn) recovery, exceed balance, expired card recovery
Edge Cases	5	Zero balance, leap year DOB, invalid account ID, spaced account ID, long name/nickname

Adversarial Test Cases (13 total)

Adversarial Category	Scenario
Amount mutation	Change amount mid-flow (500 → 200)
Account typo correction	'ACC9999. Oh wait, it's ACC1001' in same message
Number confusion	Aadhaar '1357' same as payment amount '1357 rupees'
CVV leading zeros	CVV '007' — tests string vs. int storage
Name punctuation	'Nithin Jain.' with trailing period
Zero balance insistence	User demands to pay on closed zero-balance session
Double pincode correction	'my old pincode was 400002, but current is 400001'
Micro-payment	₹0.004 rounds to zero — rejected
Cardholder name omission	No cardholder name → defaults to account name
Invalid leap year	1989-02-29 (not a leap year) → rejected by Python calendar
Mid-flow account swap	Switch ACC1001 → ACC1002 during verification, re-verify correctly
CVV-sized amount	Paying exactly ₹123 before card is entered
Exceed balance	Paying ₹9999 against ₹540 balance → re-prompt

5. Metrics Definition: What Does 'Correct' Mean?

Account Lookup

- `lookup_account()` is called exactly once per unique account ID
- Response is properly routed: 404 → retry, 200 → proceed

Verification

- `state.verified = True` after verifier returns `passed = True`
- Lockout triggers at exactly 3 failed attempts
- No sensitive field (DOB, Aadhaar, pincode) appears in any agent message

Amount

- `state.payment_amount` contains the value the user last specified (supports correction mid-flow)
- Amounts ≤ 0 or $>$ balance are rejected

Card

- All 5 card fields are populated before `process_payment()` is called
- Invalid cards are caught before the API call (Luhn check, expiry)
- Only the offending field is cleared on error

Payment

- `state.last_txn_id` is set on success
- Retryable errors only clear the specific bad field
- `Stage.ERROR` is reached after `MAX_CARD_ATTEMPTS`

6. Observed Agent Weaknesses

Based on test runs with `--llm-judge`:

Weakness	Detail
Stilted recovery phrasing	<code>payment_invalid_card_number</code> test received LLM Judge score of 6.0/10 — agent correctly rejected the card but the recovery message felt repetitive
Inference latency	Response latency is higher for complex test cases with many turns; some cases take 40–53 seconds total on a local GPU, making real-time UX dependent on model speed
Chitchat router silent failure	<code>classify_intent()</code> occasionally fails silently and defaults to <code>True</code> (treating ambiguous input as actionable), benign in most cases but can lead to a verification attempt with incomplete data. It does get autocorrected but leads to user repeating few details.
Mixed mid-flow swap edge case	Account swap mid-flow works correctly for clean inputs but edge cases where the user mixes new account ID with verification data for the old account in the same message may not be handled optimally

Minor hallucination during requesting account ID	The LLM occasionally hallucinates the account ID from 7 characters to 6 characters during the first greeting prompt. It does correct itself. But giving the user the wrong starting sentence is not good from an experience standpoint.
Slight Transparency issue	When the user decides to swap their payment amount or change the account ID, there is currently no flow coded in to let the user know that the changes have taken place. It relies on user simply trusting the system

7. Evaluation Criteria Reference

Dimension	What We Look For
System Thinking	Is there a clear, well-structured state machine? Are edge cases anticipated?
Context Handling	Does the agent track state correctly? Does it avoid re-asking questions?
Verification Logic	Is verification strict? Are retries and failure modes handled correctly?
Tool Usage	Are APIs called at the right time with correct payloads? Errors handled?
Failure Handling	Are errors communicated clearly? Does the agent recover or close gracefully?
Code Quality	Is the code readable, modular, and maintainable?
Evaluation Design	Are test cases meaningful? Is the evaluation approach thoughtful?